1 Teori och Designprinciper

Markera om följande påståenden är sanna eller falska:
(+1 för rätt svar, ingen förändring för fel svar)

En klass i C++ beksriver de attribut och metoder som varje objekt av den klass-typen skall innehålla.

- Falskt
- Sant ✔

En klass i Java beskriver bara metoderna som varje objekt av den klass-typen skall innehålla.

- Sant
- Falskt ✔

Om man programmerar mot ett interface så behöver man aldrig skapa objekt.

- Sant
- Falskt ✔

Inkapsling innebär att varje klass innehåller alla objekt av den klass-typen.

- Sant
- Falskt ✔

En superklass känner inte till vilka subklasser den har.

- Falskt
- Sant ✔

Inkapsling bidrar till Low Coupling genom att man begränsar på vilket sätt objekt kan vara kopplade till varandra.

- Falskt
- Sant ✔

High cohesion får man genom att använda sig av så kallad "klisterkod"

- Sant
- Falskt ✔

Low Coupling går ut på att varje klass skall ha så få och så välavgränsade ansvarsområden som möjligt.

- ○ Sant
- ○ Falskt ✔

En klass måste innehålla attribut för att man skall kunna skapa objekt av den typen.

- ○ Falskt ✔
- ○ Sant

Ett objekt innehåller alla metoder från "sin" klass och de klasser som den ärver ifrån.

- ○ Sant ✔
- ○ Falskt

Om ett attribut är private så ärvs det inte.

- ○ Sant
- ○ Falskt ✔

Totalpoäng: 11

2 Samlingar av Objekt

Markera om följande påståenden är sanna eller falska:
(+1 för rätt svar, ingen förändring för fel svar)

Alla objekt i en ArrayList i Java måste alltid vara av exakt samma typ.

- ○ Falskt ✔
- ○ Sant

En std::vector i C++ får inte innehålla värden av typen int eller float.

- ○ Falskt ✔
- ○ Sant

En array har begränsat med platser.

- ○ Sant ✔
- ○ Falskt

Om man till exempel skall skriva ut alla element i en std::vector så måste man veta hur många de är.

- ○ Sant
- ○ Falskt ✔

En array i C++ avslutas alltid med ett null-värde.

- ○ Falskt ✔
- ○ Sant

En Array och en ArrayList är samma sak.

- ○ Sant
- ○ Falskt ✔

Om man vill ha en samling med flera olika typer av objekt så låter man dem ärva från en gemensam basklass.

- ○ Sant ✔
- ○ Falskt

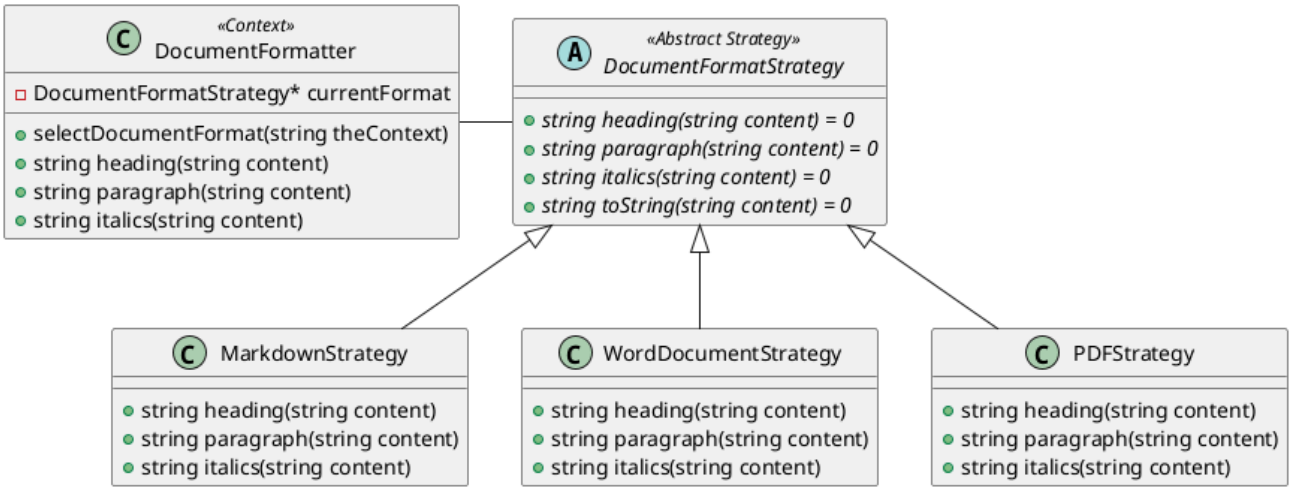
Om man vill ha olika beteenden på objekten i en samling så använder man polymorfism.

- ○ Sant ✔
- ○ Falskt

Totalpoäng: 8

3 Strategy

Betrakta nedanstående klassdiagram:



Klassdiagrammet visar en del av ett system som används för att formattera ett dokument, antingen som Markdown, som Word, eller som ett PDF-dokument.

För varje påstående nedan, markera om diagrammet stödjer påståendet (sant) eller inte stödjer påståendet (falskt).
(+1 för rätt svar, ingen förändring för fel svar)

Bara klassen DocumentFormatStrategy har implementerat metoden toString()

- ○ Sant
- ○ Falskt ✔

Klassen DocumentFormatStrategy måste känna till klasserna MarkdownStrategy, WordDocumentStrategy, och PDFStrategy

- ○ Sant
- ○ Falskt ✔

Klassen DocumentFormatter måste känna till klasserna MarkdownStrategy, WordDocumentStrategy, och PDFStrategy

- ○ Sant ✔
- ○ Falskt

i metoden DocumentFormatter::selectDocumentFormat() hittar man raden "currentFormat = new DocumentFormatStrategy();"

- ○ Falskt ✔
- ○ Sant

i metoden DocumentFormatStrategy::heading(string content) hittar man bland annat raden "return new MarkdownStrategy::heading(content);"

- Sant
- Falskt ✔

Klassen Main (inte med i diagrammet) måste ha en referens till ett objekt av typen DocumentFormatStrategy för att använda de olika formatterings-strategierna.

- Sant
- Falskt ✔

Genom Inkapsling (Encapsulation) gömmer klassen DocumentFormatter alla detaljer om hur det här systemet formatterar dokument.

- Falskt
- Sant ✔

metoden DocumentFormatter::italics(string content) innehåller raden "return currentFormat->italics(content);"

- Sant ✔
- Falskt

Totalpoäng: 8

4 Design Patterns

Markera om följande påståenden är sanna eller falska:
(+1 för rätt svar, ingen förändring för fel svar)

Strategy pattern är ett exempel på att programmera mot ett interface i stället för en klass.

- Sant ✔
- Falskt

Designmönstret Factory Method använder vi när vi vill lägga till en ny metod i en klass.

- Falskt ✔
- Sant

Designmönstret Decorator använder man när man vill utöka en existerande klass med mer avancerad funktionalitet.

- Falskt
- Sant ✔

Abstract Factory är i själva verket en variant av Strategy.

- Falskt
- Sant ✔

Abstract Factory används för att skapa rätt typ av objekt givet ett visst kontext, där resten av systemet inte behöver veta exakt vilken typ objektet är.

- Sant ✔
- Falskt

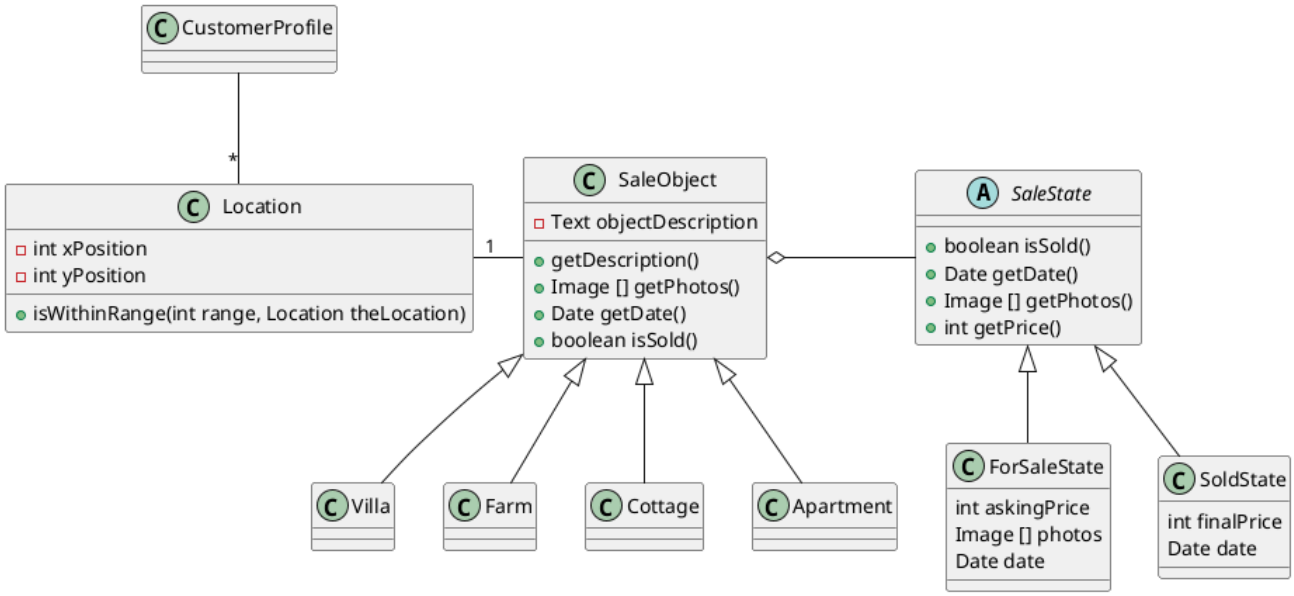
En Observable är en klass med data som andra klasser kan vara intresserade av.

- Sant ✔
- Falskt

Totalpoäng: 6

5 Klassdiagram

Betrakta nedanstående klassdiagram:



Klassdiagrammet beskriver en marknadsplats för hus (tänk Hemnet) där man kan se vilka hus som finns till salu på en karta. I sin användarprofil anger man en (eller flera) Location, och sedan kan man söka efter hus i en viss radie runt den punkten (t.ex. Ge mig alla hus inom en radie på 50 km från Karlskrona Centrum). Man kan också se vilka hus som har sålts i samma region, och vad de kostade.

För varje påstående nedan, markera om diagrammet stödjer påståendet (sant) eller inte stödjer påståendet (falskt).
(+1 för rätt svar, ingen förändring för fel svar)

När man skapar ett object av typen Location så kommer det automatiskt skapas ett objekt av typen SaleObject.

- Sant
- Falskt ✔

Det finns inga metoder för att ändra på värdena i ett Location-objekt. Har man väl skapat en :Location så går den med andra ord inte att flytta.

- Falskt
- Sant ✔

klasserna Villa, Farm, Cottage, och Apartment använder sig av Polymorfism för att hantera de skilnader som finns mellan de olika typerna av hus.

- Falskt
- Sant ✔

f:Farm består av flera åkrar som var och en har en referens till ett objekt av typen Location.

- Sant
- Falskt ✔

a:Farm, b:Cottage, och c:Apartment har alla tre en referens till samma :Location - objekt.

- Falskt
- Sant ✔

I metoden SaleObject::isSold() finns kodraden myState = new SaleState(); .

- Sant
- Falskt ✔

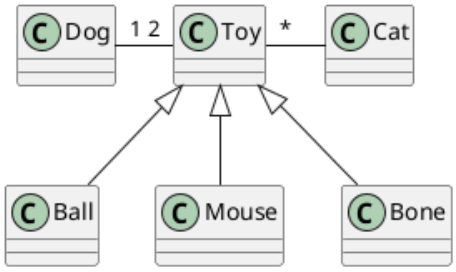
Objektet myself:CustomerProfile har referenser till två olika :Location-objekt som har samma värden på xPosition och yPosition.

- Sant ✔
- Falskt

Totalpoäng: 7

6 Relationer mellan Klasser

Betrakta nedanstående klassdiagram:



Notera att detta diagram fokuserar på relationerna mellan klasserna, och går inte in på detaljer i vilka metoder eller attribut som finns.

För varje påstående nedan, markera om diagrammet stödjer påståendet (sant) eller inte stödjer påståendet (falskt). (+1 för rätt svar, ingen förändring för fel svar)

tofflan:Cat äger b1:Ball

- Falskt
- Sant ✔

lasse:Dog äger b2:Ball och bone1:Bone

- Sant ✔
- Falskt

lasse:Dog och ludde:Dog tuggar på varsin ända av bone1:Bone

- Falskt ✔
- Sant

lasse:Dog får en ny leksak b3:Ball i julklapp. Då måste han glömma bort antingen b2:Ball eller bone1:Bone

- Falskt
- Sant ✔

odd:Cat har en m1:Mouse och en bird:Toy

- Falskt
- Sant ✔

Syskonen lau:Cat och leo:Cat leker med samma m2:Mouse

- ○ Falskt
- ○ Sant ✔

tofflan:Cat har en std::vector (Eller en java.util.Vector), där de första 42 objekten är av typen Ball

- ○ Sant ✔
- ○ Falskt

lasse:Dog och tofflan:Cat leker med b4:Ball tillsammans.

- ○ Sant ✔
- ○ Falskt

Totalpoäng: 8

Betygsgränser

Betygsgränser för den här tentan är:

Betyg	Procent	Poäng
MAX	100%	48
A	90%	43
B	80%	38
C	70%	34
D	65%	31
E	60%	29

Lycka till!